Locomotion for *Walter* (Walking Alternating Tripod for Evolutionary Research): CAD, Firmware, Optimization, and MuJoCo PPO

Howard Wang and Matthew Shiley

Engineering 90
Advisor(s): Allan Moser

April 14, 2026

Abstract

This report summarizes an Engineering 90 capstone on *Walter*, our twelve-degree-of-freedom hexapod (**W**alking **A**lternating **T**ripod for **E**volutionary **R**esearch). The mechanical structure uses **CAD-designed, 3D-printed** body, leg, and linkage parts assembled with hobby servos. Electronics include a Teensy 4.1, Pololu eighteen-channel servo controller (Maestro family), SparkFun BMI270 IMU, regulated power, and three iterations of custom *Hexapod V2* PCBs. **Proposed work** combined layered, bio-inspired control—tripod locomotion, IMU-driven reflexes, and slow on-device adaptation—with offline gait optimization and reinforcement learning in simulation. **Benefits** include interpretable gaits, repeatable artifacts, safety-aware command bounds, and policies trained without hand-written joint trajectories. **Accomplished:** open-loop optimization to firmware (`hexapod_openloop_sine`); discrete tripod baseline (`hexapod_walking_v2`); MuJoCo [1] PPO [2] via Stable-Baselines3 [3] with released evaluations. **Requirements** (`PHASES.md`) are **partially met:** hardware, CAD/mechanical documentation in this report, open-loop optimization, and RL-in-simulation are evidenced; Phase 1–4 adaptive IMU firmware paths are largely absent from the tree. Source materials are maintained at https://github.com/HowardWHSrun/WALTER. **Cost:** electronics (three BOMs) US$595.20; three PCB orders US$197.47; **grand total US$792.67** (one PCB order awaited payment at writing). Filament and fasteners are outside this sum.

1 Introduction

Reliable legged locomotion requires coordinating periodic limb motion with feedback under limited sensing and computation. Biological systems, in particular insects, organize locomotor control in *layers* that are by now standard in the comparative and robotics literature: central pattern generators (CPGs) and inter-leg coordination establish a rhythmic substrate; fast mechanosensory and

proprioceptive loops modulate timing and force in response to body perturbations; and slower processes adjust parameters over behavioral time scales [4, 5]. This project treats that decomposition not as metaphor but as a **design principle**: each major engineering choice on *Walter* was selected to preserve an explicit correspondence to this biological organization, so that hardware, firmware, and learning algorithms remain interpretable as implementations of the same layered story.

Bio-inspired correspondences. The **alternating tripod** gait used on the platform parallels the phase relationships observed in fast hexapedal locomotion: a fixed alternation of stance groups provides a stable, low-dimensional rhythmic prior before any learning is applied. **Inertial sensing** (a six-axis IMU fused for roll, pitch, and body rates) stands in for the distributed mechanoreception and body-state estimation that insects use to stabilize posture and trigger reflexive adjustments; fusion emphasizes roll and pitch over unreliable magnetic heading, consistent with how legged systems exploit gravito-inertial cues on structured terrain [6, 7]. The intended **fast** control tick (filtering, stability scoring, reflexive modulation of stride variables) and **slow** tick (windowed costs, optional parameter adaptation) mirror the separation between millisecond-scale corrective reflexes and slower tuning of gait parameters documented in adaptive locomotion models [5]. **Reinforcement learning in simulation** (PPO in MuJoCo) is used not to replace that hierarchy but to explore high-dimensional dynamics and shaped rewards while structured policies and phase cues retain a CPG-like inductive bias. **Optimization of a compact open-loop gait law** prior to closing sensor loops follows the same logic: isolate an interpretable rhythmic generator, then add sensory correction, analogous to decoupling pattern generation from sensory modulation in biological models.

Platform and scope. The physical system is *Walter*, a twelve-degree-of-freedom hexapod (WALTER: **W**alking **A**lternating **T**ripod for **E**volutionary **R**esearch). The chassis and leg mechanisms are CAD-designed, additively manufactured, and integrated with commodity servos and a custom *Hexapod V2* printed circuit board (Sec. 6.5–6.6). The target software architecture described in the project documentation specifies tripod-based gait generation, complementary-filter attitude estimation, stability scoring, reflexive modulation of gait variables, and optional slow adaptation of a low-dimensional parameter vector with nonvolatile storage; Sec. 3–5 report the degree to which each layer is realized in the current repository.

Repository. Source materials, including firmware, simulation code, and documentation, are maintained at `https://github.com/HowardWHSrun/WALTER`.

Organization of the technical work. Development proceeds along **three tracks** (Sec. 3–5): **(1)** surrogate gait optimization with deterministic export to embedded open-loop code (*complete*); **(2)** proximal policy optimization in MuJoCo with IMU-inclusive observations and tripod-structured or preset gait priors (*near complete* in simulation); and **(3)** extension of that stack with visual state for distance-aware rewards (*not yet implemented*). Sec. 2 develops the optimization and reinforcement-learning formalism shared across these tracks.

2 Mathematical background: optimization and reinforcement learning

2.1 Reduced-order gait optimization (Track 1)

2.1.1 Decision variables and bounds

The optimizer acts on a vector $\boldsymbol{\phi} \in \mathbb{R}^{25}$ matching `hexapod_no_imu_optimization.py`. Indices are concatenated as

$$\boldsymbol{\phi} = (\boldsymbol{\theta}_{\text{abd,rest}}^{\top}, \boldsymbol{\theta}_{\text{flx,rest}}^{\top}, \boldsymbol{\delta}_{\text{abd}}^{\top}, \boldsymbol{\delta}_{\text{flx}}^{\top}, f)^{\top}, \tag{1}$$

where each of $\boldsymbol{\theta}_{\text{abd,rest}}, \boldsymbol{\theta}_{\text{flx,rest}}, \boldsymbol{\delta}_{\text{abd}}, \boldsymbol{\delta}_{\text{flx}} \in \mathbb{R}^6$ are in *degrees* in the code (converted to radians internally), indexed by leg order FL, FR, ML, MR, RL, RR, and $f \in \mathbb{R}_+$ is the common gait frequency in hertz. Box constraints are implemented as `scipy.optimize.Bounds`: for example $f \in [0.4,\, 4.0]\,\text{Hz}$; resting abduction per joint in $[-25°, 25°]$; resting flexion in $[10°, 65°]$; phase shifts in $[-120°, 120°]$. These define a feasible orthotope $\boldsymbol{\phi}_{\min} \le \boldsymbol{\phi} \le \boldsymbol{\phi}_{\max}$.

2.1.2 Open-loop joint commands (sinusoidal law)

Let $\omega = 2\pi f$. For leg $\ell \in \{1, \ldots, 6\}$ with fixed tripod offset $\psi_\ell \in \{0, \pi\}$ (alternating tripod), time $t \ge 0$, and fixed amplitude tables $\bar{a}_{\text{abd},\ell}$, $\bar{a}_{\text{flx},\ell}$ (defaults in code, not decision variables), the commanded abduction and flexion angles are

$$\phi_{\text{abd},\ell}(t) = \theta_{\text{abd,rest},\ell} + \bar{a}_{\text{abd},\ell}\, \sin\!\big(\omega t + \psi_\ell + \delta_{\text{abd},\ell}\big), \tag{2}$$

$$\phi_{\text{flx},\ell}(t) = \theta_{\text{flx,rest},\ell} + \bar{a}_{\text{flx},\ell}\, \sin\!\big(\omega t + \psi_\ell + \delta_{\text{flx},\ell}\big) - \Delta_{\text{lift},\ell}(t), \tag{3}$$

where $\Delta_{\text{lift},\ell}$ implements swing-phase foot clearance (subtracting flexion when the abduction sine is positive), identical in structure to the firmware `LIFT_SWING_DEG_PEAK` term. Velocities $\dot{\phi}_{\text{abd},\ell}$, $\dot{\phi}_{\text{flx},\ell}$ are obtained by differentiation and feed the planar surrogate dynamics.

2.1.3 Surrogate planar dynamics and simulation

The reduced model integrates an eight-dimensional state

$$\mathbf{x}(t) = (x,\, \dot{x},\, z,\, \dot{z},\, \phi_{\text{roll}},\, \dot{\phi}_{\text{roll}},\, \phi_{\text{pitch}},\, \dot{\phi}_{\text{pitch}})^{\top}, \tag{4}$$

with $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\phi})$ assembled from lumped masses, soft ground contact, leg thrust and support forces derived from $\phi_{\text{abd},\ell}$, $\phi_{\text{flx},\ell}$ and contact weights, and linear damping on body motion and attitude rates (`body_ode` in the script). For fixed $\boldsymbol{\phi}$, the trajectory is obtained by `solve_ivp` (Runge–Kutta) over $t \in [0, T]$ with T the simulation horizon (`sim_duration_s`), sampled at N time points (`sample_count`).

2.1.4 Scalar objective (explicit weighted sum)

Let a subscript "ss" denote statistics evaluated on a *steady-state* suffix of the horizon (the code uses the last 60% of samples for oscillation metrics and mean speed). Write $\bar{v}_x$ for mean forward velocity $\mathbb{E}_{\mathrm{ss}}[\dot{x}]$, and RMS values $\mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{roll}}]$, $\mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{pitch}}]$, $\mathrm{RMS}_{\mathrm{ss}}[z]$. Let $q_{\mathrm{servo}} \geq 0$ be a slack on peak torque proxy exceeding a threshold (0.85 of stall in the implementation), and $\sigma_{\mathrm{contact}}$ the time mean of the per-timestep standard deviation of contact weights across legs. With nonnegative weights $w_\cdot$ matching `OptimizationConfig`, the implemented cost is

$$\begin{aligned}\mathcal{J}(\boldsymbol{\phi}) = & -w_{\mathrm{fwd}}\,\bar{v}_x + w_{\mathrm{back}}\left[\max(0, -\bar{v}_x)\right]^2 \\ & + w_{\mathrm{rp}}\Big(\mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{roll}}]^2 + \mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{pitch}}]^2\Big) + w_z\,\mathrm{RMS}_{\mathrm{ss}}[z]^2 \\ & + w_{\mathrm{servo}}\,q_{\mathrm{servo}}^2 + w_{\mathrm{contact}}\,\sigma_{\mathrm{contact}}^2 + w_{\mathrm{reg}}\,\|\boldsymbol{\phi} - \boldsymbol{\phi}_0\|_{\mathrm{scaled}}^2,\end{aligned} \tag{5}$$

where $\boldsymbol{\phi}_0$ is the default parameter vector and $\|\cdot\|_{\mathrm{scaled}}$ averages coordinatewise squared normalized deviation from the box midpoint (Tikhonov-style regularization keeping $\boldsymbol{\phi}$ in a "reasonable" neighborhood). Optional terms penalize shortfall below a target mean speed if `target_forward_speed_mps` $>$ 0. Infeasible simulations return a large penalty (10^6). Equation (5) is the precise surrogate counterpart of the abstract sum $\sum_k w_k c_k$ used earlier: here each c_k is spelled out.

2.1.5 Optimization algorithm and restarts

The problem solved in software is

$$\min_{\boldsymbol{\phi}} \; \mathcal{J}(\boldsymbol{\phi}) \quad \text{subject to} \quad \boldsymbol{\phi}_{\min} \leq \boldsymbol{\phi} \leq \boldsymbol{\phi}_{\max}. \tag{6}$$

L-BFGS-B (`scipy.optimize.minimize`, method `L-BFGS-B`) approximates curvature of $\mathcal{J}$ with a limited-memory inverse Hessian and projects each iterate onto the box. Each evaluation of $\mathcal{J}$ requires one numerical integration of the surrogate. The outer loop runs R random restarts (`restarts`): one start at $\boldsymbol{\phi}_0$ and $R-1$ uniform samples in the box; the best $\boldsymbol{\phi}^\star$ by lowest $\mathcal{J}$ is retained. Limits `maxiter`, `maxfun`, and `ftol` cap work per restart.

The **code path** from optimization to firmware is deterministic: $\boldsymbol{\phi}^\star$ is serialized to `no_imu_optimization_summary.` `sync_gait_params_from_json.py` emits `optimized_gait_params.h`; `hexapod_openloop_sine.ino` evaluates the same `leg_commands()` map, applies bench degree-to-pulse gains and clamps, and streams servo targets. Thus optimization-to-code is *algebraic parity*, not a learned policy.

2.2 Markov decision processes and returns

Reinforcement learning treats legged control as a **Markov decision process** (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$: states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transition density $P(s'|s,a)$ from the simulator (MuJoCo), reward

$r(s,a)$, and discount $\gamma \in (0,1)$. A stochastic policy $\pi_\theta(a|s)$ with parameters θ induces trajectories $\tau = (s_0, a_0, r_0, \ldots)$. The **return** from time t is the discounted sum $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$. The **state-value** and **action-value** functions under π_θ are

$$V^{\pi_\theta}(s) = \mathbb{E}[G_t \mid s_t = s], \qquad Q^{\pi_\theta}(s,a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]. \tag{7}$$

The **advantage** $A^{\pi_\theta}(s,a) = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s)$ measures how much better action a is than the policy average in state s. Policy-search methods estimate gradients of the expected return $\eta(\theta) = \mathbb{E}[G_0]$; the **policy gradient theorem** motivates updates proportional to $\nabla_\theta \log \pi_\theta(a|s)\, A(s,a)$, but raw Monte Carlo estimates have high variance.

2.3 Generalized advantage estimation (GAE) and the value baseline

Practical actor–critic algorithms maintain a **critic** $V_\psi(s) \approx V^{\pi_\theta}(s)$ with parameters ψ. One-step **TD residuals** are $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$. **Generalized advantage estimation** (GAE) combines multi-step returns with an exponential weighting by $\lambda \in [0,1]$ (see Schulman *et al.* on GAE; used inside SB3's PPO [2, 3]):

$$\hat{A}_t = \sum_{\ell=0}^{\infty} (\gamma\lambda)^\ell \, \delta_{t+\ell}, \tag{8}$$

which trades bias (relying on V_ψ) against variance (longer rollouts). Stable-Baselines3 uses this form when `gae_lambda` is set in the PPO configuration.

2.4 Proximal policy optimization (PPO)

PPO [2] stabilizes policy updates by limiting how far θ moves from the behavior policy θ_{old} that collected the batch. Define the **probability ratio**

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{9}$$

For continuous actions, π_θ is typically Gaussian; r_t is the ratio of densities. The **clipped surrogate objective** for a batch of timesteps is

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_t\big)\Big], \tag{10}$$

with clip range $\epsilon \in (0,1)$ (often 0.2). Large policy steps that inflate r_t when $\hat{A}_t > 0$ (or shrink r_t when $\hat{A}_t < 0$) are clipped so the effective objective stops increasing, reducing catastrophic updates common in vanilla policy gradients.

Let $\hat{R}_t$ denote a target return for the critic (e.g., n-step return or GAE-backed value target). With

coefficients $c_v, c_e > 0$, the **composite loss** maximized in practice is the negative of

$$\mathcal{L}_{\text{total}}(\theta, \psi) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v \, \mathbb{E}_t\big[(V_\psi(s_t) - \hat{R}_t)^2\big] - c_e \, \mathbb{E}_t\big[H(\pi_\theta(\cdot \mid s_t))\big], \tag{11}$$

where H is differential entropy for Gaussian policies. Minimizing $\mathcal{L}_{\text{total}}$ corresponds to ascending $\mathcal{L}^{\text{CLIP}}$ while fitting the critic and discouraging collapse of exploratory variance. Updates use mini-batches over several epochs on each rollout buffer; hyperparameters (learning rate, rollout length `n_steps`, batch size, γ, λ, ϵ, c_v, c_e) are set in YAML under `hexapod_training_new/configs/`.

3 Track I: Surrogate optimization and firmware translation (complete)

Status: done. This track implements Sec. 2.1: the parameterization of $\boldsymbol{\phi}$, sinusoidal `leg_commands()`, the eight-state surrogate ODE, the explicit cost (5), box-constrained L-BFGS-B with random restarts, and deterministic export of $\boldsymbol{\phi}^\star$ to firmware. It isolates gait timing and amplitude before closing any sensor loop on hardware.

Deliverables. `hexapod_no_imu_optimization.py` (L-BFGS-B, constraints, random restarts); `no_imu_optimization_summary.json`; `sync_gait_params_from_json.py`; `hexapod_openloop_sine.ino` with `optimized_gait_params.h`. Surrogate metrics and $\boldsymbol{\phi}^\star$ appear in Tables 6–7; the MuJoCo training model is illustrated in Fig. 4.

4 Track II: PPO with IMU-informed observations and preset tripod structure (near complete)

Status: near complete (simulation and training stack; hardware deployment partial). Here the policy is $\pi_\theta(a|s)$ with s containing **proprioception** (joint positions and velocities), **base kinematics** from the simulator, and—in IMU-focused configs—accelerometer and gyroscope readings (or their low-pass / complementary-filter equivalents) so that roll, pitch, and body rates enter the observation stack, matching the intended Walter firmware sensing. The **preset tripod structure** is enforced by one or more of: (i) a fixed gait clock or phase features in s; (ii) structured policies (e.g., sCPG or phase-parameterized heads) that bias action toward tripod alternation; or (iii) reward terms that penalize deviation from productive tripod coordination. PPO updates θ using Eqs. (8)–(10) on MuJoCo rollouts.

Relation to mathematics. The environment implements P and r; the agent maximizes $\eta(\theta)$ subject to PPO's trust-region-like clipping. IMU channels shrink the *partial observability* gap between simulation and the physical BMI270 stack; training runs such as `imu6_001` / `imu_bno10_001` in `hexapod_training_new/runs/` document this observation mode. Released evaluations (Table 8)

remain predominantly proprioceptive / path-following; extending hardware parity for Track II requires flashing a policy or residual layer while the Maestro executes tripod-compatible targets.

5 Track III: Preset tripod, IMU, and vision for distance-traveled PPO (planned)

Status: not done. The goal is to augment the state with **visual odometry** or pose estimates from an overhead or onboard camera so the agent (or a critic) can optimize **distance traveled** along a body or world axis with less simulator privilege. Let $p_t \in \mathbb{R}^2$ be horizontal position from integration of visual motion or fiducial tracking, and let $\Delta_t = \|p_t - p_0\|$ or forward displacement $x_t - x_0$. A natural reward increment is

$$r_t = \alpha\,(x_{t+1} - x_t) - \beta\,\|\omega_t\|^2 - \rho\,\mathrm{penalty}_{\mathrm{fall}} - \kappa\,\mathrm{TV}(a)_t, \tag{12}$$

with IMU rates ω_t penalizing harsh attitude changes, and optional terms for staying upright. The **observation** becomes $s_t = [s_t^{\mathrm{IMU/proprio}}, f_t]$ where f_t is a vector of visual features or estimated pose; PPO then applies the same surrogate (10) with expanded input dimension.

Open problems. Sim-to-real on vision (latency, calibration, lighting); defining r_t from *estimated* versus ground-truth displacement; and maintaining the preset tripod prior while learning residuals. This track is the logical successor once Track II policies are stable on Walter with IMU feedback.

Table 1: Summary of the three development tracks for *Walter*.

Track	Technical core	Status
I	Surrogate cost $\mathcal{J}(\boldsymbol{\phi})$ (5); L-BFGS-B; JSON→`.h`→ open-loop firmware	Complete
II	PPO $\mathcal{L}^{\mathrm{CLIP}}$ (10); GAE (8); IMU in s; tripod prior / structured policy	Near complete (simulation; hardware partial)
III	Same as II plus visual features or pose f_t; rewards from estimated distance / odometry	Not started

6 Technical Discussion

6.1 Biological and control context

The Introduction motivates *Walter* as a layered, bio-inspired architecture; this subsection records additional literature touchpoints used in detailed design. Decentralized feedback atop rhythmic pattern generators remains the standard modeling idiom for insect locomotion [4, 5]. Complementary or quaternion attitude filters address accelerometer–gyro fusion when magnetometers are untrust-

worthy [6, 8]; hexapod posture control on irregular terrain likewise foregrounds body orientation and angular velocity [7]. Disturbance-triggered gait transitions provide a classical benchmark for reflex and supervisory layers [9].

6.2 Layered architecture (specification)

The intended runtime stack, documented in `docs/ARCHITECTURE.md`, uses two timescales. A **fast tick** (∼10–20 ms) reads the IMU, fuses attitude, computes a stability score S, applies reflex adjustments, steps the gait interpolator, clamps commands, and streams twelve servo setpoints. A **slow tick** (each gait cycle) aggregates windowed RMS of S, runs adaptation, and saves the best parameter vector. The complementary blend

$$\alpha \in (0,1), \quad \theta_{\text{roll}} \leftarrow \alpha(\theta_{\text{roll}} + \omega_x \Delta t) + (1-\alpha)\,\theta_{\text{roll,accel}}, \tag{13}$$

(and analogously for pitch) is the nominal attitude update.

6.3 Open-loop sinusoidal gait (implemented)

For the no-IMU milestone, each leg follows resting abduction and flexion angles plus sinusoidal variation at common frequency f, with tripod group B offset by π from group A. This is the same law optimized in Sec. 2.1 and delivered in Track I (Sec. 3). Amplitudes follow default tables shared by Python and `optimized_gait_params.h`.

6.4 MuJoCo and reinforcement learning (simulation)

High-fidelity training uses Gymnasium + MuJoCo [1] with PPO [2] as implemented in Stable-Baselines3 [3]; the mathematical objective is Eq. (10) with advantages (8). Tracks II–III (Sec. 4–5) specialize the observation and reward; YAML configs set γ, λ, learning rate, network widths, PD gains, and terrain. A representative static render of the simulation model appears in Fig. 4 (Sec. 9.2).

6.5 Mechanical design: CAD and 3D-printed parts

Walter's chassis and legs are built from **parts designed in CAD** and produced by **additive manufacturing** (3D printing): a central body, six leg assemblies, and two-degree-of-freedom linkages per leg (abduction and flexion axes) that mate to the servo output splines. The design prioritizes a repeatable assembly for twelve actuated joints, clearance for wiring, and attachment of the electronics tray and battery. Figure 1 shows an overall render; Figure 2 shows the body, leg, and linkage components used in the assembly. Fasteners and servo hardware complete the mechanical

stack; material usage (filament mass) and printed-part cost are not included in the dollar totals in Tables 3–5.

6.6 Electronics, Hexapod V2 PCB, and safety

The bill of materials matches purchased components: Teensy-class MCU (SparkFun 16771), Pololu 1354 eighteen-channel driver, buck converter, BMI270 via Qwiic cabling, dual-cell LiPo and charger, terminals, and servos. Custom *Hexapod V2* PCBs (revision v5 dated 2026-04-03 on the latest order) integrate power and signal routing for Walter's stack; Figure 3 shows a board photo and schematic excerpt. `docs/FIRMWARE.md` records UART to the servo controller, channel ordering FL/FR/ML/MR/RL/RR with abduction then flexion per leg, and pulse mapping $u_k = \mathrm{clamp}(c_k + d_k q_k)$ (typical bounds 600–2400 μs). Open-loop firmware applies degree-to-command gains and clamps before streaming compact-protocol targets; these bench gains are not outputs of the reduced-order optimizer.

7 Requirements and Constraints

7.1 Source of requirements

Engineering requirements are traced to phase exit criteria in `docs/PHASES.md` (Phases 1–4: IMU telemetry, reflexes, cautious mode and recovery, online adaptation), supplemented by safety and interface constraints in `docs/FIRMWARE.md` and the open-loop scope in `firmware/hexapod_openloop_sine/README.m` `PHASES.md` serves as the team's consolidated requirements-and-constraints specification for firmware behavior.

7.2 Summary: met vs. not met

Met (repository evidence): discrete tripod baseline (`hexapod_walking_v2`); open-loop optimizer and summary JSON/CSV; firmware `hexapod_openloop_sine` with JSON-to-header script; calibration helpers; design, architecture, and firmware documentation; CAD/3D-printed mechanical design for *Walter* documented in this report (Figs. 1–2); MuJoCo/PPO training and evaluation artifacts under `hexapod_training_new/` and `e90_repo/`.

Partial / not in version control: Phase 1 IMU telemetry sketch path may be absent as `hexapod_imu_test/`; Phases 2–4 adaptive firmware (`hexapod_adaptive/`) not present; full on-device learning as described in `docs/on_device_learning/` remains planning relative to the checked-in firmware tree.

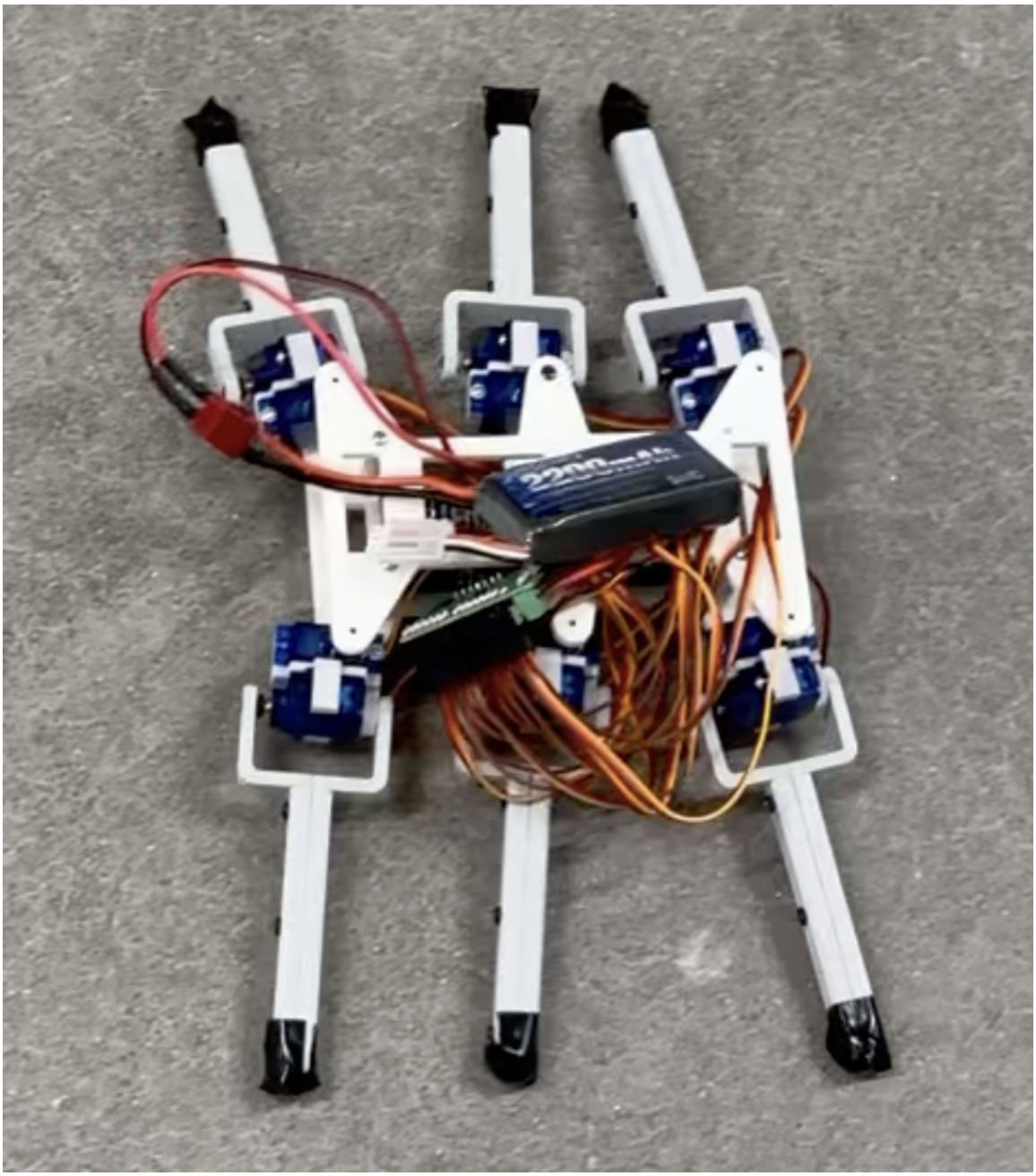
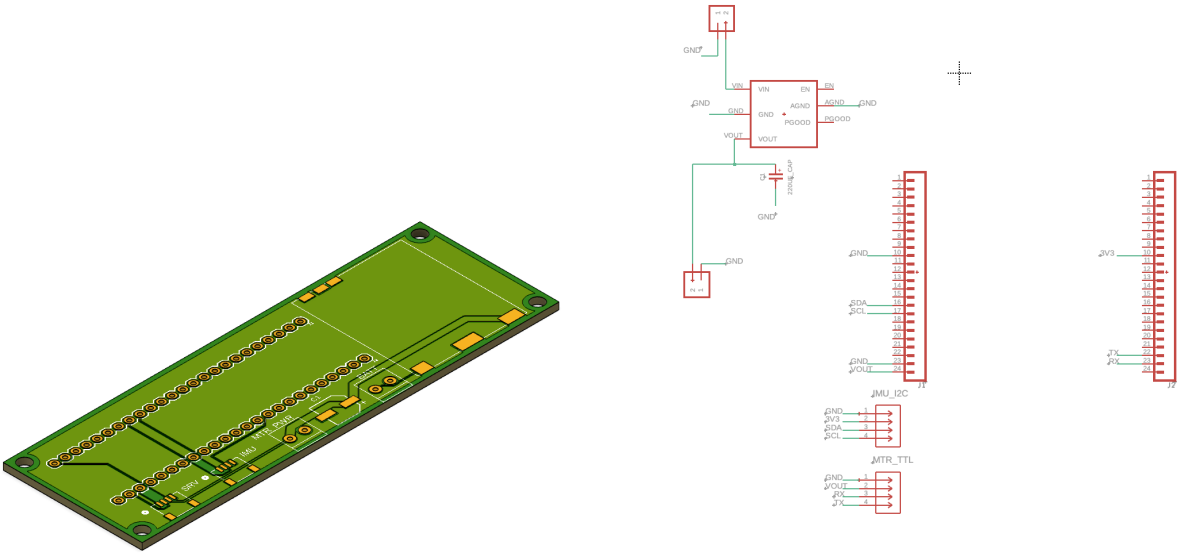


Figure 1: Walter: full-assembly render of the twelve-DOF hexapod (CAD; Walking Alternating Tripod for Evolutionary Research).

(a) Body (b) Leg assemblies (c) Linkage

Figure 2: CAD-designed 3D-printed subassemblies for Walter: chassis, legs, and per-leg linkage.



(a) Assembled PCB (Hexapod V2) (b) Schematic (excerpt)

Figure 3: Custom electronics for Walter: Hexapod V2 board and schematic.

Table 2: Requirement traceability. Evidence points to repository artifacts.

ID	Requirement	Status	Evidence / notes
R1	Phase 1: IMU read, complementary filter, USB telemetry ∼50 Hz; gait unchanged	Not met (in repo)	`PHASES.md`; folder may be missing
R2	Phase 2: postural reflexes, clamps, serial tuning, EEPROM	Not met (in repo)	`ARCHITECTURE.md` pseudocode
R3	Phase 3: stability score, cautious mode, push recovery, fall freeze	Not met (in repo)	Documented only
R4	Phase 4: five-parameter adaptation, cost J, SPSA/bandit, EEPROM	Not met (in repo)	Documented only
R5	Hardware: MCU + eighteen-channel driver + servos + documented map + PCBs + Walter mechanical CAD/3D parts	Met	BOM, Figs. 1–3, `FIRMWARE.md`
R6	Safety: pulse clamps; parameter slew (adaptive layer)	Partial	Clamps in v2/openloop; slew in spec
R7	Open-loop: optimizer/firmware parity; JSON export	Met	Optimizer, `sync_gait_params_from_json.py`
R8	Simulation: MuJoCo env, PPO training, documented evals	Met	`hexapod_training_new/`, `e90_repo/released_checkpoints/`

7.3 Project cost

Tables 3–4 summarize **purchased** electronics (three identical platforms) and **fabrication** costs for three PCB/stencil orders. Table 5 gives the grand total. Servo counts in Table 3 are per platform (eight units listed per BOM line); completing twelve DOF per robot may require additional servos beyond those lines if not already stocked.

8 Methods

8.1 Mechanical assembly (*Walter*)

Printed body and leg components are fastened per the CAD design; each leg receives two servos driving abduction and flexion through the linkage. The custom PCB mounts to the chassis and connects to the Pololu driver and Teensy; power and ground respect the buck converter and screw terminals from the BOM. Bench alignment uses `set_zero` and side-only test sketches before closed-loop or open-loop gait tests.

Table 3: Electronics bill of materials, **one platform** (USD). The team purchased three identical sets.

Part	Description	Vendor	Qty	Unit	Line
SparkFun 16771	Microcontroller	DigiKey	1	37.20	37.20
Pololu 1354	18-channel servo driver	DigiKey	1	46.95	46.95
DFRobot DFR1202	Step-down buck converter	DFRobot	1	12.90	12.90
SparkFun BMI270	6-DoF IMU	DigiKey	1	18.50	18.50
SparkFun 11855	2×7.4 V LiPo + charger	Amazon	1	27.99	27.99
SparkFun 14417	Female Qwiic connector	DigiKey	2	0.60	1.20
SparkFun 17259	100 mm Qwiic cable	DigiKey	1	1.95	1.95
SparkFun 17261	Qwiic cable to female header	DigiKey	1	1.95	1.95
Amphenol YO0201500000G	1×2 screw-down terminal	DigiKey	2	1.28	2.56
DFRobot SER0039	Servo motor	DigiKey	8	5.90	47.20
Subtotal	**one platform**				**198.40**
Electronics total	**three platforms (×3)**				**595.20**

Table 4: PCB and stencil orders, *Hexapod V2* (USD order totals including merchandise, shipping, duties/taxes, and sales tax where charged by the vendor).

Order ID	Description / notes	Status	Total
Y4-11911448A	PCB prototype (5 pcs) + SMT stencil; v5_2026-04-03	Awaiting payment	74.24
W2026030804205410	PCB + stencil shipment	Shipped	68.26
W2026021709554129	PCB + stencil (earlier order)	Shipped	54.97
PCB subtotal			**197.47**

Table 5: Grand total project cost (USD) for reported purchases and fabrication.

Category	Amount
Electronics (three platforms)	595.20
PCB fabrication and stencils (three orders)	197.47
Grand total	**792.67**

8.2 Baseline discrete gait

`firmware/hexapod_walking_v2/hexapod_walking_v2.ino` implements a deterministic tripod state machine with smoothed keyframes (`moveSmoothTo`).

8.3 Reduced-order simulation optimization

`hexapod_brain_roadmap copy/ode_optimization_no_imu/hexapod_no_imu_optimization.py` integrates the surrogate model with `scipy.integrate.solve_ivp`, evaluates the objective on a steady-state window, and calls `scipy.optimize.minimize` (L-BFGS-B). Outputs include `no_imu_optimization_s` and optional CSV/plots.

8.4 Parameter sync and open-loop firmware

From `firmware/hexapod_openloop_sine/`, run `python3 sync_gait_params_from_json.py` to regenerate `optimized_gait_params.h`. The sketch evaluates the sinusoidal law at `INTERP_DELAY_MS` (default 10 ms), applies startup frequency ramping, maps degrees to integer commands, clamps, and sends Pololu compact-protocol targets on `Serial1`.

8.5 Bench testing and calibration

`firmware/set_zero/` and side-only test sketches support calibration. The open-loop README documents sign and scale tuning when feet drag or the robot shuffles backward.

8.6 MuJoCo PPO training and evaluation

Training uses `hexapod_training_new/train.py` with YAML configs under `configs/`; runs write `training_summary.yaml`, checkpoints, and logs under `runs/<run_name>/`. Evaluation uses `evaluate.py` with the same config as training; the `e90_repo/released_checkpoints/` tree archives representative `.zip` weights and `evaluations/<name>/REPORT.md` summaries for slides and reporting.

9 Results

9.1 Reduced-order optimization (surrogate model)

Table 6 summarizes `hexapod_brain_roadmap copy/ode_optimization_no_imu/no_imu_optimization_summary` These metrics characterize the *surrogate* model; hardware performance requires bench validation of gains, centers, and directions.

Table 6: Optimizer-reported metrics (committed summary JSON).

Quantity	Value
Gait frequency f	1.80 Hz
Mean forward speed (steady window)	0.234 m/s
Total displacement (rollout)	1.86 m
Roll RMS	0.440 rad
Pitch RMS	~ 0 (near zero in file)
Vertical position RMS	0.086 m
Peak servo usage ratio (proxy)	0.152
Contact balance std. dev.	0.388
Dominant oscillation frequency (estimated)	3.54 Hz
Objective cost $\mathcal{J}(\boldsymbol{\phi}^\star)$	-228.38

Table 7 lists the corresponding optimized parameter vector $\boldsymbol{\phi}^\star$ (rest angles, phase shifts, frequency, and swing lift) exported to `optimized_gait_params.h`. Flexion phase shifts sit at the lower bound ($-90°$), trading stride phasing for the surrogate's forward-speed and regularization terms; hardware tuning may move off that bound.

Table 7: Optimized decision variables $\boldsymbol{\phi}^\star$ from L-BFGS-B (same JSON as Table 6). Angles in degrees; leg order FL, FR, ML, MR, RL, RR.

Quantity	Value
Gait frequency f	1.800 Hz
Abduction rest $\boldsymbol{\theta}_{\mathrm{abd,rest}}$	(10, 10, 0, 0, -10, -10)
Flexion rest $\boldsymbol{\theta}_{\mathrm{flx,rest}}$	(32.00, 32.00, 34.00, 34.00, 32.00, 32.00)
Abduction phase $\boldsymbol{\delta}_{\mathrm{abd}}$	$\approx \mathbf{0}$ (magnitude $< 10^{-6}$ deg)
Flexion phase $\boldsymbol{\delta}_{\mathrm{flx}}$	(-90, -90, -90, -90, -90, -90)
Swing lift amplitude	12.0 deg (`lift_swing_deg_peak`)

9.2 MuJoCo simulation (high-fidelity model)

Track II–III training uses Gymnasium with the MuJoCo asset `hexapod_training_new/assets/hexapod.xml`: checkerboard ground, twelve torque-controlled joints, and RK4 integration at 0.002 s (see configs for frame skipping). Figure 4 is an **offscreen MuJoCo render** (software `Renderer`, no interactive viewer): the model was advanced under gravity with zero actuator commands until the body settled on the plane, then a fixed camera was used. The view documents the mesh, materials, and lighting used for PPO rollouts; it is not a policy-controlled frame. Interactive inspection is available via `scripts/live_mujoco_viewer.py` with a trained checkpoint.

Figure 4: MuJoCo screenshot of the Walter hexapod model used for PPO training and evaluation (post-settle, zero control; offscreen render).

9.3 MuJoCo PPO evaluations (released checkpoints)

Table 8 quotes short evaluation protocols documented in `e90_repo/released_checkpoints/evaluations/`. Rewards are **not** comparable across configs (different shaping and scaling); forward displacement and path length provide a common physical readout within each experiment family.

Table 8: Representative PPO evaluation means (300 env steps per episode unless noted; see each REPORT.md).

Checkpoint / run	Task / policy	Fwd. $+x$	2D path len.	Note
`mlp_clean_run005_best`	Circle path; MLP 256×256 PPO; `mlp_run_005` (301k steps)	0.18 m	0.27 m	Mean reward ≈ 680
`terrain_low_shake_run001_best`	Flat / rough / steps; MLP PPO	0.33 m mean over 3 ep.	—	Flat/rough ≈0.47 m fwd.; steps harde

The `pd_velocity_window_run018_best` checkpoint documents an sCPG + PD + velocity-window experiment; its released short eval shows small net displacement—useful as an archival structured-policy datapoint rather than a primary locomotion benchmark (see that folder's `REPORT.md`).

9.4 Requirements and cost (results view)

Open-loop, hardware documentation, PCB procurement, and simulation requirements (R5–R8) are supported by artifacts. Phase 1–4 adaptive firmware requirements (R1–R4) are **not** satisfied in the repository snapshot. Purchased engineering spend for this accounting is **US$792.67** (Tables 3–5).

10 Discussion

Context. Track I gives an interpretable, verifiable optimization-to-code pipeline. Track II applies the PPO mathematics of Sec. 2 to rich MuJoCo dynamics with IMU-capable observation stacks and tripod-structured policies; Track III would add visual state for distance-centric rewards but is not yet built. We separate **simulation** results from **hardware** validation: closing the loop on II–III requires IMU integration, possible camera/odometry, power-under-load tests, and honest reporting against `PHASES.md` exit criteria.

What went well. A named platform (*Walter*) with traceable CAD and 3D-printed parts; shared command laws between Python and C++; JSON-to-header automation; a stable servo-driver and custom PCB stack; a growing library of RL configs, runs, and released evaluations.

Limitations. Model mismatch and servo saturation can invalidate predicted speeds; manual degree-to-pulse scaling; open-loop policies do not reject disturbances. RL rewards are task-specific—cross-comparing raw reward across configs is misleading.

Lessons for future E90 students. Freeze a logging format and baseline gait before adding sensors; budget time for bench tuning after simulation; document PCB spins and BOM multiples early; keep a written trace from optimizer or checkpoint to deployed constants.

11 Conclusion

We presented *Walter* with CAD/PCB/cost documentation; a mathematical treatment of surrogate optimization (L-BFGS-B), MDPs, GAE, and PPO; and **three tracks**: (1) optimization-to-code (*complete*); (2) PPO with IMU and preset tripod structure (*near complete* in sim); (3) IMU + vision + distance PPO (*planned*). Materials are at https://github.com/HowardWHSrun/WALTER. Major takeaways: interpretable optimization complements learned policies; IMU and eventually vision shrink the sim-to-real observation gap; requirement tracing must distinguish which track is deployed on hardware.

Acknowledgements

We thank our advisor, Allan Moser, for guidance on the project. We thank Swarthmore Engineering and shop or lab staff who supported fabrication and testing. No funding sources beyond the Engineering department are declared for this accounting.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 5026–5033, 2012.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations." Journal of Machine Learning Research, 2021.

[4] Y. O. Aydin *et al.*, "Mechanosensory control of locomotion in animals and robots: Moving forward," *Biomimetics*, vol. 8, no. 4, p. 211, 2023. PMC10445419.

[5] G. Chen *et al.*, "Adaptive locomotion control of a hexapod robot via bio-inspired learning," *Frontiers in Neurorobotics*, vol. 15, p. 627157, 2021.

[6] R. G. Valenti, I. Dryanovski, and J. Xiao, "Keeping a good attitude: A quaternion-based orientation filter for IMUs and MARGs," *Sensors*, vol. 15, no. 8, pp. 19302–19330, 2015.

[7] L. Bai *et al.*, "Development and implementation of a new approach for posture control of a hexapod robot to walk in irregular terrains," *Robotica*, vol. 37, no. 12, pp. 2225–2238, 2019.

[8] R. Zhang *et al.*, "Attitude estimation algorithm of portable mobile robot based on complementary filter," *Micromachines*, vol. 12, no. 11, p. 1373, 2021.

[9] A. Johnson, K. D. Gregg, and D. E. Koditschek, "Disturbance detection, identification, and recovery by gait transition in legged robots," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 3627–3634, 2010.

A Public repository

The public project URL is https://github.com/HowardWHSrun/WALTER. Appendix paths below refer to the local Engineering 90 workspace layout at the time of writing; the GitHub repository is intended to subsume or mirror these components for reproducibility.

B Repository map (selected local paths)

- **Adaptive firmware and docs:** `RL Temp/adaptive_ondevice_hexapod/README.md`, `docs/DESIGN.md`, `docs/ARCHITECTURE.md`, `docs/PHASES.md`, `docs/FIRMWARE.md`
- **Firmware sketches:** `firmware/hexapod_walking_v2/`, `firmware/hexapod_openloop_sine/`, `firmware/set_zero/`, side-only tests
- **Reduced-order optimization:** `hexapod_brain_roadmap copy/ode_optimization_no_imu/`
- **MuJoCo PPO training:** `RL Temp/hexapod_training_new/` (`train.py`, `configs/`, `runs/`)
- **Released checkpoints and evals:** `RL Temp/e90_repo/released_checkpoints/`
- **Presentation notes:** `Presentation/E90_MidSemester_SpeakerScript.tex` (and PDF)
- **This folder (figures):** `walter_assembly.png`, `Body.png`, `Legs.png`, `Linkage.png`, `pcb_board.png`, `pcb_schematic.png`, `mujoco_hexapod_sim.png` (plus originals with longer filenames)